

Lecture 8:

Backpropagation and Gradient Descent

Vegard Høghaug Larsen

GRA4160: Predictive modelling with machine learning

February 25th 2026

Plan for today:

- ▶ **Topic:** *Backpropagation and Gradient Descent* - understanding how neural networks learn
- ▶ **Key Concepts:** Gradient descent variants, computational graphs, the chain rule, backpropagation algorithm
- ▶ **Approach:** Start from first principles, then live-code a simple automatic differentiation engine
- ▶ **Goal:** Build intuition on how gradients are computed and used to train models

Example: Logistic regression

What is Gradient Descent?

- ▶ An optimization algorithm to **minimize a loss function** by iteratively moving in the direction of **steepest descent** (negative gradient).
- ▶ **Update rule:** For a parameter w ,

$$w \leftarrow w - \eta \nabla_w L,$$

where η is the learning rate (step size).

- ▶ **Objective:** find parameters that minimize the loss (error) on training data, by continuously updating weights opposite to the gradient.

What is the gradient?

- ▶ By gradient we mean the **partial derivatives** of the loss with respect to the parameters.
- ▶ **Gradient:** with two parameters w_1 and w_2 , the gradient of the loss with respect to a parameter w_i is given by

$$\nabla_{w_i} L = \frac{\partial L}{\partial w_i}.$$

- ▶ **Gradient vector:** the gradient vector is given by

$$\nabla_w L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2} \right).$$

- ▶ The gradient vector points in the direction of the steepest ascent of the loss function.

- ▶ **Batch Gradient Descent:** Compute gradient on the entire dataset before each update
- stable but can be very slow for large data.
- ▶ **Stochastic Gradient Descent (SGD):** Compute gradient on a single example at a time - much faster updates but noisier (updates may fluctuate).
- ▶ **Mini-Batch Gradient Descent:** Compute gradient on a small batch of examples (e.g. 32 or 64) - a compromise that offers faster convergence than full-batch and more stability than pure SGD.
- ▶ In practice, mini-batches are most common (e.g. 32 samples per update) to leverage vectorized computation and achieve good convergence.

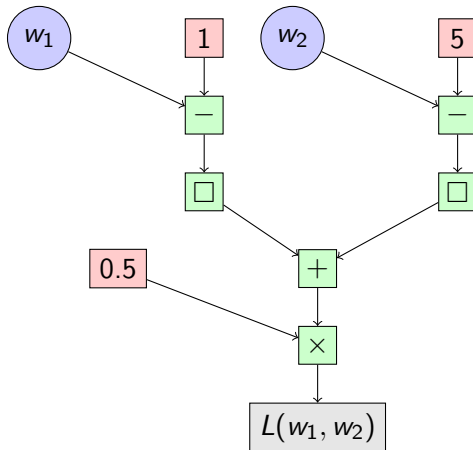
A computational graph is a directed graph representing the flow of data through a sequence of operations.

- ▶ Nodes are operations or variables; edges show dependencies.
- ▶ Example: The expression

$$L(w_1, w_2) = ((w_1 - 1)^2 + (w_2 - 5)^2) \times 0.5$$

can be represented as a graph of simpler operations (subtraction, square, addition, multiply).

Computational Graphs: a loss function



- ▶ In the above example the loss function L is a function of the parameters w_1 and w_2 . In a neural network, the loss is a function of all the weights and biases in the network.
- ▶ We can then look at Neural networks as large computational graphs: input flows forward through layers to compute output, and we can trace how each weight influences the final loss.
- ▶ Deep learning frameworks (e.g. PyTorch, TensorFlow) build these graphs during the forward pass, recording operations and their inputs/outputs.
- ▶ This sets the stage for **automatic differentiation**.
- ▶ Automatic differentiation is a technique to compute the gradient of a function with respect to its inputs automatically using the **chain rule**.

Chain Rule - The Engine of Backpropagation

- ▶ **Chain Rule (Calculus):** If $y = f(u)$ and $u = g(x)$, then $\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$. For a composition of many functions, gradients multiply along the chain.
- ▶ In a computational graph, the chain rule allows us to propagate gradients backward: the gradient of a node's output w.r.t. some input equals the product of gradients along the path from input to output.
- ▶ **Local Gradients:** Each operation knows how its output changes w.r.t. its inputs. Backpropagation uses these local derivatives and the chain rule to get global gradients.
- ▶ **Key idea: Backpropagation** boils down to applying the chain rule repeatedly on a graph of computations.

What is Backpropagation?

- ▶ Backpropagation = backward propagation of errors - the core algorithm that computes the gradient of the loss with respect to all parameters in the model.
- ▶ It is essentially an efficient application of the chain rule on the computational graph of the model's forward pass. It allocates portions of the output error to each weight by traversing the graph in reverse.
- ▶ **Forward pass:** compute outputs and loss using current weights.
- ▶ **Backward pass:** starting from the loss, propagate the error gradient backward through each operation (layer) to find $\frac{\partial L}{\partial w}$ for every weight w .
- ▶ The result is the gradient vector given by $\nabla_{\theta} L$ (partial derivatives of loss w.r.t. each parameter collected in a vector θ). We use this to update weights (e.g. using gradient descent).

Backpropagation – Step by Step

Consider the network

$$f_{\theta}(x) = \sigma\left(\sum_k \beta_k h_k(x) + b^{(2)}\right) \quad \text{with} \quad h_k(x) = \sigma\left(\sum_j w_{kj} x_j + b_k^{(1)}\right).$$

For a single data point (x, y) , define the loss as $L = \frac{1}{2}(y - f_{\theta}(x))^2$. σ is the sigmoid function.

Backpropagation – Step by Step

Consider the network

$$f_{\theta}(x) = \sigma\left(\sum_k \beta_k h_k(x) + b^{(2)}\right) \quad \text{with} \quad h_k(x) = \sigma\left(\sum_j w_{kj} x_j + b_k^{(1)}\right).$$

For a single data point (x, y) , define the loss as $L = \frac{1}{2}(y - f_{\theta}(x))^2$. σ is the sigmoid function.

1. **Forward Pass:** Compute the prediction $f_{\theta}(x)$ and store intermediate values (e.g. $h_k(x)$ and $z_k = \sum_j w_{kj} x_j + b_k^{(1)}$).
2. **Output Gradient:** Initialize with $\frac{\partial L}{\partial L} = 1$ and compute the gradient w.r.t. the output:

$$\frac{\partial L}{\partial f_{\theta}(x)} = -(y - f_{\theta}(x)).$$

3. **Backprop through Output Layer:** For each output weight β_k , compute

$$\frac{\partial L}{\partial \beta_k} = \frac{\partial L}{\partial f_{\theta}(x)} \cdot h_k(x).$$

Backpropagation – Step by Step (Cont.)

4. **Backprop through Hidden Layer:** For each hidden weight w_{kj} (connecting input j to hidden unit k), recall that

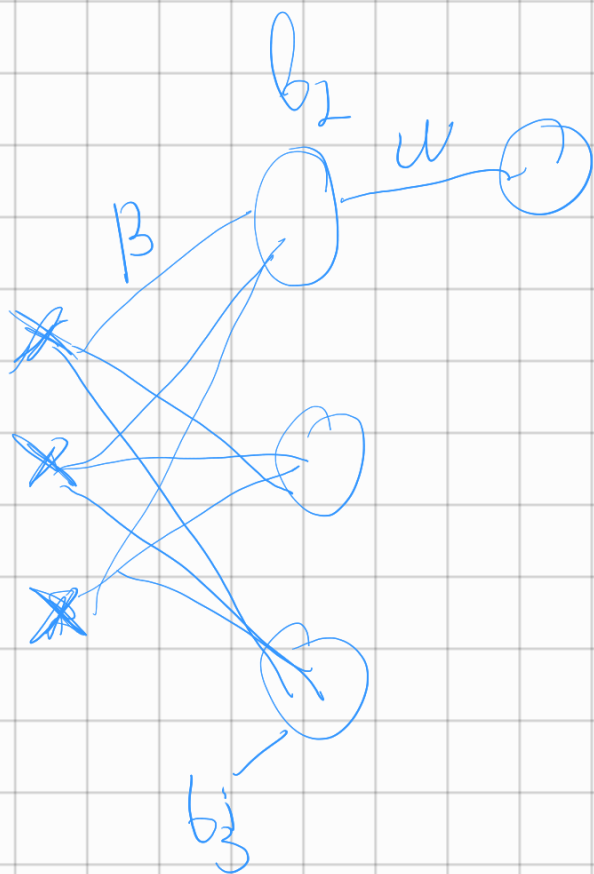
$$z_k = \sum_j w_{kj} x_j + b_k^{(1)} \quad \text{and} \quad h_k(x) = \sigma(z_k).$$

Then, the gradient is given by

$$\frac{\partial L}{\partial w_{kj}} = \frac{\partial L}{\partial f_{\theta}(x)} \cdot \beta_k \cdot \sigma'(z_k) \cdot x_j.$$

5. **Extend to Deeper Layers:** For networks with more layers, apply the chain rule recursively, propagating the error backwards.
6. **Gradient Updates:** Once the full gradient $\nabla_{\theta} L$ is computed, update each parameter using gradient descent:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L.$$



Training with Backprop + Gradient Descent

Combine backprop with an iterative training loop: for each epoch (and for each batch of data):

- ▶ Forward pass: compute predictions and loss
- ▶ Backward pass: compute gradients via backpropagation
- ▶ Update weights: $w \leftarrow w - \alpha \nabla_w L$ for all weights (using the chosen gradient descent variant)
- ▶ Batch GD: use all training data to compute an average gradient, then update once per epoch.
- ▶ Stochastic GD: update for each training example (one forward/backward per example).
- ▶ Mini-batch GD: update for each small batch (common in practice, e.g. 32 samples gives a good balance).

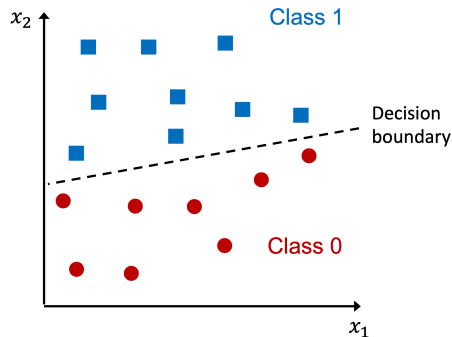
Iterate for many epochs - the loss typically decreases and the model improves.

Backpropagation ensures each weight is nudged in the direction that most reduces the error.

Outcome: The network's weights gradually converge to values that produce low loss (i.e., the network learns from the data).

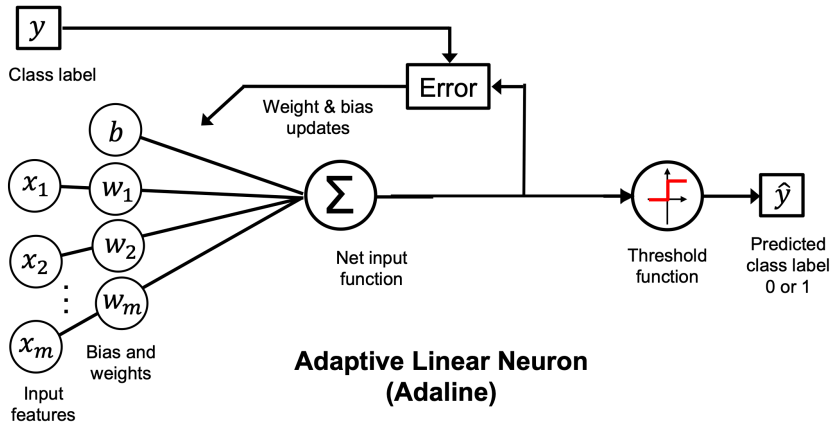
ADALINE for binary classification

Supervised learning for binary classification: Linear decision boundary



- ▶ Binary classification task with two classes: 0 and 1
- ▶ We can construct a linear decision boundary
- ▶ We use a threshold function for discriminating among the two classes
- ▶ The decision function takes as input a linear combination of the features, that is $\mathbf{z} = \mathbf{w}^\top \mathbf{x} + \mathbf{b}$ (net input)

ADALINE: ADaptive Linear NEurons



Step 1. Updating the weights

ADALINE rule

1. We initialize the weights and bias;
2. We compute the NET INPUT: $z^{(i)} = \mathbf{w}^\top x^{(i)} + b$;
3. We compute the error in the training step BY $y^{(i)} - z^{(i)}$, where now $y^{(i)} \in \{0, 1\}$ but $z^{(i)}$ takes values in all $\mathbb{R}$;

Since we want to minimize the difference $y^{(i)} - z^{(i)}$:

- ▶ For those $x^{(i)}$ corresponding to $y^{(i)} = 1$, the weights $\mathbf{w}$ and b needs to *push* $z^{(i)}$ as *closed as possible to 1*;
- ▶ For those $x^{(i)}$ corresponding to $y^{(i)} = 0$, the weights $\mathbf{w}$ and b needs to *push* $z^{(i)}$ as *closed as possible to 0*.

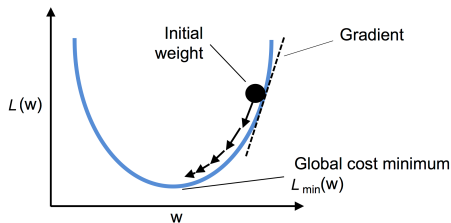
Step 2. We use a threshold function to make the final prediction

- ▶ At the end of the training, we get $z^{(i)}$ which will not be 0 or 1 as we need!! (we need to identify a class);
- ▶ The $z^{(i)}$ will take some values close to 0 or 1;
- ▶ We need then to "transform" them by applying a threshold function;

Step 2. We use a threshold function to make the final prediction

- ▶ At the end of the training, we get $z^{(i)}$ which will not be 0 or 1 as we need!! (we need to identify a class);
- ▶ The $z^{(i)}$ will take some values close to 0 or 1;
- ▶ We need then to "transform" them by applying a threshold function;
- ▶ A possible threshold function: $\hat{y}^{(i)} = \begin{cases} 1 & \text{if } z^{(i)} \geq 0.5 \\ 0 & \text{if } z^{(i)} < 0.5 \end{cases}$;
- ▶ REMEMBER THAT THE THRESHOLD FUNCTION IS ONLY USED TO MAKE THE FINAL PREDICTION!

The optimization algorithm via gradient descent



- We use again the **mean squared error (MSE)** loss function:

$$L(\mathbf{w}, b) = \frac{1}{2n} \sum_{i=1}^n \left(y^{(i)} - z^{(i)} \right)^2$$

where now the residuals are computed between the observations and the net inputs

- We take a step in the opposite direction of the gradient
- The step size is determined by the learning rate and by the slope of the gradient

The rule

- ▶ By gradient descent, we update the model parameters by taking a step in the opposite direction of the gradient of the loss function:

$$\mathbf{w} = \mathbf{w} + \Delta \mathbf{w}$$

$$\text{with } \Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L(\mathbf{w}, b)$$

$$b = b + \Delta b$$

$$\text{with } \Delta b = -\eta \nabla_b L(\mathbf{w}, b)$$

- ▶ $\nabla_{\mathbf{w}} L(\mathbf{w}, b)$ is the vector whose components are the partial derivatives of the loss function with respect to each weight w_j :

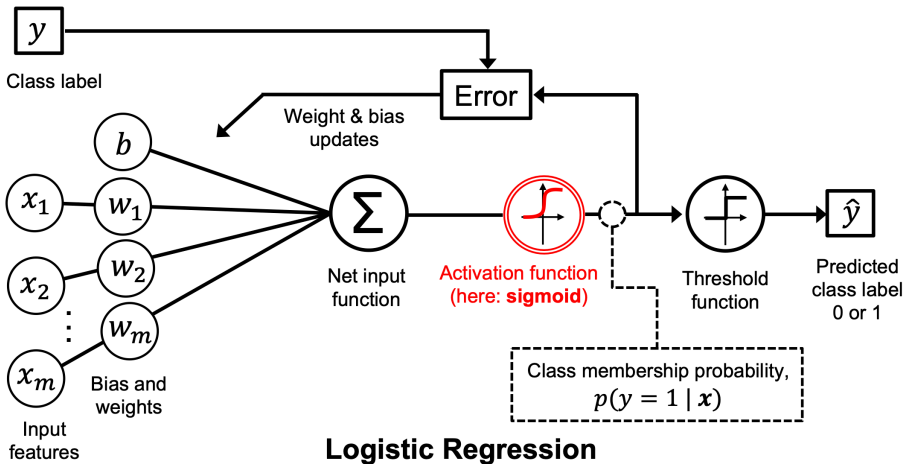
$$\nabla_{\mathbf{w}} L(\mathbf{w}, b) = \left(\frac{\partial L}{\partial w_1} \quad \frac{\partial L}{\partial w_2} \quad \cdots \quad \frac{\partial L}{\partial w_m} \right)^{\top} \text{ with } \frac{\partial L}{\partial w_j} = -\frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - z^{(i)} \right) x_j^{(i)};$$

- ▶ $\nabla_b L(\mathbf{w}, b)$ corresponds to the partial derivative of the loss function with respect to the bias b :

$$\nabla_b L(\mathbf{w}, b) = \frac{\partial L}{\partial b} = -\frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - z^{(i)} \right).$$

Logistic Regression

The logistic regression algorithm



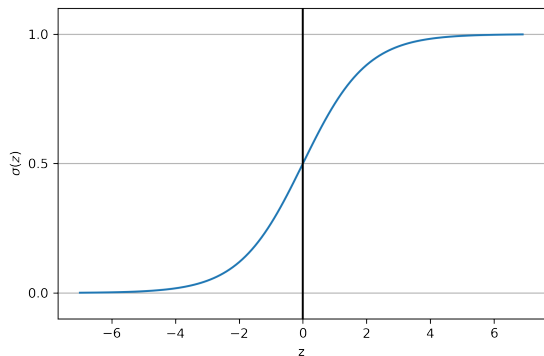
What is this similar to? What is/are the difference(s)?

The sigmoid function

We define the **logistic sigmoid function**, or simply **sigmoid function** by:

$$\sigma(z) := \frac{1}{1 + e^{-z}}$$

where $z = \mathbf{w}^\top \mathbf{x} + b$ is the net input.



Notice that:

- ▶ For z approaching $+\infty$, $\sigma(z)$ approaches 1;
- ▶ For z approaching $-\infty$, $\sigma(z)$ approaches 0;
- ▶ Hence $\sigma : \mathbb{R} \rightarrow [0, 1]$ with intercept $\sigma(0) = 0.5$.

The (same) threshold function (as Adaline)

The predicted probability can then simply be converted into a binary outcome via a threshold function:

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{if } \sigma(z) < 0.5 \end{cases}$$

Remark

There are many applications where we are interested in both the predicted class labels and in **the estimation of the class-membership probability** (the output of the sigmoid function prior to applying the threshold function).

Examples:

- ▶ Weather forecasting: we are interested not only to predict whether it will rain on a particular day, but also to report the chance of rain.
- ▶ Medicine: logistic regression can be used to predict the chance that a patient has a particular disease given certain symptoms.

What about the learning process?

- ▶ We need to define an objective function that we seek to minimize;
- ▶ The idea is that we want to **maximize the likelihood**, namely the probability of classifying the dataset correctly, given some parameters $\mathbf{w}$ and b ;
- ▶ This is equivalent to minimize the **CROSS-ENTROPY loss function**:

$$L(\mathbf{w}, b) := \sum_{i=1}^n \left[-y^{(i)} \log(\sigma(z^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(z^{(i)})) \right].$$

Example: $n = 1$

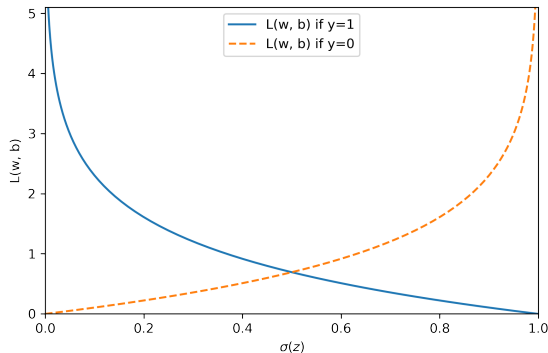
$$L(\mathbf{w}, b) = -y^{(1)} \log(\sigma(z^{(1)})) - (1 - y^{(1)}) \log(1 - \sigma(z^{(1)}))$$

We notice that:

$$L(\mathbf{w}, b) = \begin{cases} -\log(\sigma(z^{(1)})) & \text{if } y^{(1)} = 1 \\ -\log(1 - \sigma(z^{(1)})) & \text{if } y^{(1)} = 0 \end{cases}.$$

The cross-entropy function explained

$$L(\mathbf{w}, b) = \begin{cases} -\log(\sigma(z^{(1)})) & \text{if } y^{(1)} = 1 \\ -\log(1 - \sigma(z^{(1)})) & \text{if } y^{(1)} = 0 \end{cases}$$



- ▶ Blue continuous line: the loss approaches 0 if we correctly predict that an example belongs to class 1
- ▶ Orange dashed line: the loss also approaches 0 if we correctly predict $y = 0$
- ▶ If the prediction is wrong, the loss goes toward infinity
- ▶ **We penalize wrong predictions with an increasingly larger loss!**

The logistic regression algorithm can be obtained starting from the Adaline implementation:

1. By substituting the MSE loss function with the cross-entropy loss function
2. By changing the linear activation function with the sigmoid function

► What about the gradient for the optimization step?

The logistic regression algorithm can be obtained starting from the Adaline implementation:

1. By substituting the MSE loss function with the cross-entropy loss function
 2. By changing the linear activation function with the sigmoid function
-
- ▶ What about the gradient for the optimization step?
 - ▶ Using calculus, we can show (optional exercise at home) that **the parameter updates via gradient descent are equal to the ones for Adaline**, namely the gradient step is unchanged!

The gradient step (unchanged)

- By **gradient descent**, we update the model parameters by taking a step in the opposite direction of the gradient of the loss function:

$$\mathbf{w} = \mathbf{w} + \Delta \mathbf{w}$$

$$\text{with } \Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L(\mathbf{w}, b)$$

$$b = b + \Delta b$$

$$\text{with } \Delta b = -\eta \nabla_b L(\mathbf{w}, b)$$

- $\nabla_{\mathbf{w}} L(\mathbf{w}, b)$ is the vector whose components are the partial derivatives of the loss function with respect to each weight w_j :

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b) = \left(\frac{\partial L}{\partial w_1} \quad \frac{\partial L}{\partial w_2} \quad \cdots \quad \frac{\partial L}{\partial w_m} \right)^\top \text{ with } \frac{\partial L}{\partial w_j} = -\frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)};$$

- $\nabla_b L(\mathbf{w}, b)$ corresponds to the partial derivative of the loss function with respect to the bias b :

$$\nabla_b L(\mathbf{w}, b) = \frac{\partial L}{\partial b} = -\frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - \sigma(z^{(i)}) \right).$$

Coding Demo: Building an Automatic Differentiation Engine

- ▶ We will now build a simple automatic differentiation engine from scratch (in code) to illustrate backpropagation in action.
- ▶ This engine will construct a computational graph during the forward pass and then automatically compute gradients via backprop (chain rule) in the backward pass.
- ▶ We'll use it to optimize a small example by gradient descent - seeing how the code mirrors the math.